KUKU / PROTOTYPE BRIEF

Legal Intake Assistant

Problem 2 - AI lawyer | One user, one step: organize a rental or eviction concern before a lawyer conversation

The problem I chose

A tenant facing a rental or eviction dispute may arrive at a lawyer conversation with a long story but no clear dates, payment trail, lease details, or evidence list. I chose the intake step: turn the person's own account into a concise note that is easier to review.

> **Output:** stated facts, an ordered timeline, missing facts and focused questions, relevant retrieved legal material when available, and an evidence checklist. The assistant does not decide whether an eviction is lawful or predict an outcome.

How it works

1. User story
Tenant describes the issue and known facts in the CLI.

2. Retrieve
Chroma searches local legal chunks; the agent retrieves 3 passages per query.

3. Draft
Groq receives the question and retrieved passages with a cautious system prompt.

4. Reviewable note
Answer separates stated facts, gaps, sources, and evidence to gather.

`build_knowledge_base.py` indexes locally supplied JSON and compressed JSONL files into Chroma, using 1,600-character chunks with 180-character overlap. `agent.py` can perform up to three retrieval steps. `agent_multirag.py` searches complementary query formulations. `llm_client.py` calls Groq. Conversation history lasts only for the current process.

Why this scope and the assumptions

- The project already contains a CLI, retrieval pipeline, and Indian legal material; the intake note makes one step concrete and checkable.
- Assume the user's question concerns an Indian rental or eviction issue, the supplied facts are incomplete, and the local corpus may not cover every question.
- Unknown details stay unknown. The assistant must not invent lease terms, notice rules, deadlines, evidence, or legal outcomes.
- Kuku's assignment input pack is confidential: it is used locally and excluded from the public repository.

A lawyer or trained reviewer remains responsible for legal conclusions. This prototype gives general information, not legal advice.

QUALITY, SCALE, AND LIMITS

How I would know it works

Measure usefulness and factual safety with real intake work, not model confidence.

Pilot evaluation

Run a pilot with 20 consented, anonymized rental-intake cases. A lawyer or trained reviewer compares each note with the original facts and retrieved sources. Record:

Fidelity
Key facts retained; no material invented facts.

Organization
Timeline order and useful missing questions.

Evidence
Relevance of retrieved sources and checklist.

Workflow value
Reviewer usability and intake-preparation time saved.

Proposed launch gate: at least 18 of 20 notes rated usable, zero material invented facts in the reviewed set, and measurable time saved against the team's current intake. These are targets, not achieved results. If a source is missing or retrieval is weak, the assistant should say so and hand the question to a person.

Scaling across Kuku's legal work

Reuse the same intake workflow for another issue type only after adding a reviewed source collection, issue-specific questions, and a human-approved evaluation set. Store jurisdiction and source-version metadata, re-index on controlled updates, and monitor retrieval relevance, unsupported-claim rate, latency, and cost. For higher volume, add managed vector storage, session-level access controls, and a reviewer queue. The current multi-query path uses about two model calls per question; iterative retrieval may use up to three. Actual spend depends on token volume and current provider pricing.

Current run and limitations

The repository and code are prepared, but this submission run did not return an AI answer: the user-provided run reports Groq HTTP 403, “Access denied. Please check your network settings.” This environment also has no usable Python interpreter, so successful raw outputs could not be generated here. The exact reported error is included with three independent test prompts and a capture script. No quality target is claimed as met.

- CLI only; no deployed web app, authentication, persistent case files, or automated tests.
- Corpus coverage, chunking, embeddings, and model output can all limit answer quality.
- Do not send confidential inputs to an external model without authorization.

Prototype

github.com/Akash-M-100/AI-LAWYER- | **Run steps: repository README**

The submission form still needs the applicant's name, roll number, and branch. The repository link is the prototype; the assignment input pack is not included.